

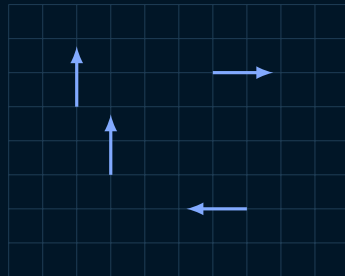
Simulador de Vientos

Autómata de Gas Reticular · LBM-D2Q4

Dinámica de fluidos y termodinámica mediante
Diseño Orientado a Datos (DOD)

Grupo 1

Ingeniería de Sistemas



Resumen Ejecutivo

Concepto

Simulador bidimensional de fluidos y termodinámica sobre un Autómata Celular discreto estricto.

Paradigma central

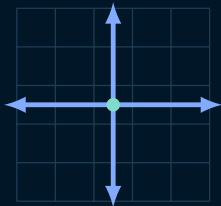
Diseño Orientado a Datos (DOD) frente al enfoque OOP tradicional o Lagrangiano puro.

Rendimiento

Física resuelta mediante operaciones de tensores paralelos con NumPy.

Complejidad

Garantizada $\mathcal{O}(N \cdot M)$ por fotograma, eliminando bucles de iteración espacial.



Topología de Von Neumann

Cuadrícula 2D con 4 direcciones ortogonales (N, S, E, O). Sin diagonales: la energía cinética viaja exclusivamente por ejes cartesianos.

Estados de Celda Q

Superposición de tensores: vectorial, material, presión y temperatura.

Función de Transición δ

Evolución paralela $t \rightarrow t + 1$ mediante transformaciones matriciales simultáneas.

Estructura de Datos / Los Tensores

```
def __init__(self, w, h):  
    self.w, self.h = w, h  
    self.materials = np.zeros((w, h), dtype=np.int8)  
    self.static_pressure = np.zeros((w, h), dtype=np.int16)  
    self.temperature = np.zeros((w, h), dtype=np.float32)  
    self.tracers = np.zeros((w, h), dtype=np.int8)  
    self.wind = np.zeros((4, w, h), dtype=np.int16)
```

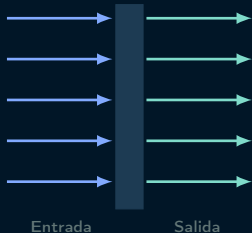
Tensor	Shape	Propósito
wind	$(4, W, H)$	Vectores de flujo
materials	(W, H)	Mapa de entorno
static_pressure	(W, H)	Fuentes de presión
temperature	(W, H)	Energía térmica
tracers	(W, H)	Partículas de polvo

Sin objetos por celda: memoria contigua en C (NumPy) para máxima localidad de caché L1/L2.

Reglas de Interacción y Entorno

El autómata evalúa la topología mediante máscaras lógicas sobre el tensor **materials**:

ID	Tipo	Comportamiento
0	Aire	Flujo libre y colisiones.
1	Muro	Bounce-back 180° perfecto.
2	Follaje	Resta -1 al vector por frame.
3	Sumidero	Aniquila viento y presión.



Motor Físico / Pipeline de `update_step()`

En cada paso discreto $t \rightarrow t + 1$ el autómata ejecuta **6 fases matriciales**:

1. **Gradiente de presión**

Genera viento desde $\Delta P \geq 2$

2. **Propagación (Streaming)**

`np.roll` desplaza vectores

3. **Fricción estocástica**

Entropía: 2 % de decaimiento por frame

4. **Colisiones HPP**

Choques frontales rotan 90°

5. **Termodinámica**

Advección + difusión + convección

6. **Trazadores Lagrangianos**

Polvo arrastrado por el viento dominante

Ecuación maestra:

$$V_{t+1} = V_t - c \nabla P$$

Fase 1 / Gradiente de Presión Barométrica

```
mask_up   = air & (self.static_pressure - p_up   >= 2)
mask_down = air & (self.static_pressure - p_down >= 2)
mask_left = air & (self.static_pressure - p_left >= 2)
mask_right = air & (self.static_pressure - p_right >= 2)

self.wind[DIR_UP][mask_up]   = np.clip(self.wind[DIR_UP][mask_up]   + 1, 0, 3)
self.wind[DIR_DOWN][mask_down] = np.clip(self.wind[DIR_DOWN][mask_down] + 1, 0, 3)
self.wind[DIR_LEFT][mask_left] = np.clip(self.wind[DIR_LEFT][mask_left] + 1, 0, 3)
self.wind[DIR_RIGHT][mask_right] = np.clip(self.wind[DIR_RIGHT][mask_right] + 1, 0, 3)
```

Mecanismo:

Si una celda de aire supera a su vecina en ≥ 2 unidades de presión, se inyecta un cuanto de viento en dirección contraria al gradiente.

Implementación:

`np.roll` desplaza el tensor de presión para comparar vecinos sin bucles.

Fase 2 / Propagación Vectorial y Rebote en Muros

```
n_up = np.roll(self.wind[DIR_UP], -1, axis=1)
n_down = np.roll(self.wind[DIR_DOWN], 1, axis=1)
n_left = np.roll(self.wind[DIR_LEFT], -1, axis=0)
n_right = np.roll(self.wind[DIR_RIGHT], 1, axis=0)

# Extraemos el viento que acaba de penetrar la máscara del muro
bounce_up = np.where(walls, n_up, 0)
bounce_down = np.where(walls, n_down, 0)
bounce_left = np.where(walls, n_left, 0)
bounce_right = np.where(walls, n_right, 0)

# Borramos las celdas del muro para que no absorban nada
n_up[walls] = 0
n_down[walls] = 0
n_left[walls] = 0
n_right[walls] = 0

# Devolvemos el rebote a la celda de aire en dirección opuesta
n_down += np.roll(bounce_up, 1, axis=1)
n_up += np.roll(bounce_down, -1, axis=1)
n_right += np.roll(bounce_left, 1, axis=0)
n_left += np.roll(bounce_right, -1, axis=0)

self.wind[DIR_UP] = np.clip(n_up, 0, 3)
self.wind[DIR_DOWN] = np.clip(n_down, 0, 3)
self.wind[DIR_LEFT] = np.clip(n_left, 0, 3)
self.wind[DIR_RIGHT] = np.clip(n_right, 0, 3)
```

Streaming:

Cada canal vectorial se desplaza un paso en su dirección mediante rotación matricial.

Bounce-back:

El viento que penetra un muro se devuelve en dirección opuesta conservando magnitud.

Fase 4 / Colisiones de Fluidos — Modelo HPP

```
up = self.wind[DIR_UP].copy()
down = self.wind[DIR_DOWN].copy()
left = self.wind[DIR_LEFT].copy()
right = self.wind[DIR_RIGHT].copy()
pairs_y = np.minimum(up, down); pairs_x = np.minimum(left, right)
space_x = 3 - np.maximum(left, right); space_y = 3 - np.maximum(up, down)
rotate_y_to_x = np.clip(pairs_y - pairs_x, 0, space_x)
rotate_x_to_y = np.clip(pairs_x - pairs_y, 0, space_y)
delta = rotate_x_to_y - rotate_y_to_x
self.wind[DIR_UP] = np.clip(up + np.where(air, delta, 0), 0, 3)
self.wind[DIR_DOWN] = np.clip(down + np.where(air, delta, 0), 0, 3)
self.wind[DIR_LEFT] = np.clip(left - np.where(air, delta, 0), 0, 3)
self.wind[DIR_RIGHT] = np.clip(right - np.where(air, delta, 0), 0, 3)
```

Hardy–Pomeau–de Pazzis:

Los choques frontales transfieren energía al eje perpendicular, conservando masa y momento total.

Branchless:

Sin condicionales if/else. La aritmética tensorial reemplaza toda lógica de colisión.

Fase 5 / Termodinámica — Advección, Difusión y Convección

```
T = self.temperature.copy()
T_air = T * air # temperatura activa solo en aire
T_adv = T.copy()
weight = 0.25
for ch, (shift, axis) in enumerate([(1, 1), (1, 1), (-1, 0), (1, 0)]):
    intensity = self.wind[ch].astype(np.float32) / 3.0
    src = np.roll(T_air + intensity, shift=shift, axis=axis)
    T_adv += src * weight
    T_adv -= T_air * intensity * weight

# Laplaciano normalizado: solo vecinos de aire, sin atravesar muros
T_n = np.roll(T_air, -1, axis=0); T_s = np.roll(T_air, 1, axis=0)
T_e = np.roll(T_air, -1, axis=1); T_w = np.roll(T_air, 1, axis=1)
air_f = air.astype(np.float32)
a_n = np.roll(air_f, -1, axis=0); a_s = np.roll(air_f, 1, axis=0)
a_e = np.roll(air_f, -1, axis=1); a_w = np.roll(air_f, 1, axis=1)
n_sum = T_n*a_n + T_s*a_s + T_e*a_e + T_w*a_w
n_cnt = a_n + a_s + a_e + a_w
lap = np.where(n_cnt > 0, n_sum - n_cnt * T_air, 0.0)

T_new = T_adv + 0.04 * lap
T_new *= 0.97
self.temperature[air] = np.clip(T_new[air], -100.0, 100.0)
self.temperature[~air] = 0.0 # muros no acumulan calor

# CORRECCIÓN DE BUG: La Convección térmica (Térmicas) solo redirige flujos existentes
hot = (self.temperature > 5) & air
cold = (self.temperature < -5) & air
prob_conv = np.random.random((self.w, self.h)) < 0.3

swap_up = hot & prob_conv & (self.wind[DIR_DOWN] > 0)
self.wind[DIR_DOWN][swap_up] = np.clip(self.wind[DIR_DOWN][swap_up] - 1, 0, 3)
self.wind[DIR_UP][swap_up] = np.clip(self.wind[DIR_UP][swap_up] + 1, 0, 3)

swap_down = cold & prob_conv & (self.wind[DIR_UP] > 0)
self.wind[DIR_UP][swap_down] = np.clip(self.wind[DIR_UP][swap_down] - 1, 0, 3)
self.wind[DIR_DOWN][swap_down] = np.clip(self.wind[DIR_DOWN][swap_down] + 1, 0, 3)
```

Advección:

El viento arrastra la temperatura.

Difusión:

Laplaciano discreto enmascarado por muros.

$$T_{t+1} = T_{adv} + \alpha \nabla^2 T$$

Convección:

Calor $\Rightarrow$ corriente ascendente.
Frío $\Rightarrow$ descendente.

Fase 6 / Cinemática Lagrangiana — Trazadores de Polvo

```
vx = self.wind[DIR_RIGHT].astype(np.int16) - self.wind[DIR_LEFT].astype(np.int16)
vy = self.wind[DIR_DOWN].astype(np.int16) - self.wind[DIR_UP].astype(np.int16)
new_tracers = np.zeros_like(self.tracers)
for dx in [-1, 0, 1]:
    for dy in [-1, 0, 1]:
        if dx == 0 and dy == 0: continue
        mask = (np.sign(vx)==dx) & (np.sign(vy)==dy) & (self.tracers==1)
        if np.any(mask):
            moved = np.roll(np.roll(mask, shift=dx, axis=0), shift=dy, axis=1)
            # Si intenta atravesar muro, anulamos el movimiento y forzamos rebote inverso
            hit_wall = moved & walls
            moved[hit_wall] = False
            bounced = np.roll(np.roll(hit_wall, shift=-dx, axis=0), shift=-dy, axis=1)

            new_tracers = np.logical_or(new_tracers, moved)
            new_tracers = np.logical_or(new_tracers, bounced)

# Mantener partículas en reposo si no hay viento
no_wind_mask = (vx == 0) & (vy == 0) & (self.tracers == 1)
new_tracers = np.logical_or(new_tracers, no_wind_mask)

new_tracers[walls] = 0
new_tracers[self.materials == MAT_SINK] = 0
self.tracers = new_tracers.astype(np.int8)
```

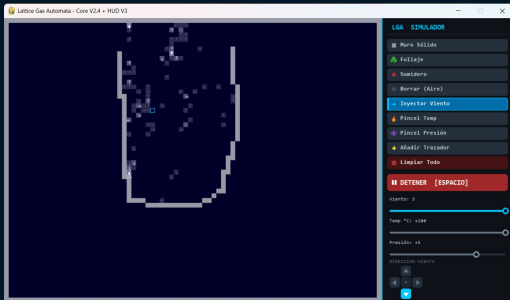
Partículas sin masa:

Consultan el vector dominante en su posición y se desplazan diagonalmente.

Rebote elástico:

Si el destino es un muro, la partícula retrocede a su posición original.

Implementación Visual / Frontend Desacoplado



- ▶ **Mapas de densidad**
Presión en espectro azul/morado.
Temperatura en rojo/cian.
- ▶ **Vectores de viento**
Modulados por magnitud sobre el campo escalar.
- ▶ **Render vectorizado**
surfarray: numpy → GPU sin iterar celdas.
- ▶ **HUD interactivo**
Muros, sumideros, pinceles de presión y temperatura.

¿Por qué no hay diagonales?

Propio del álgebra D2Q4. Introducir diagonales requeriría D2Q9, rompiendo la conservación de masa discreta sin interpolación.

¿Cruce sin colisión?

Artefacto de paridad: en espacio-tiempo discreto, partículas en $x = 0$ y $x = 1$ intercambian posiciones. Solo colisionan al compartir índice en el instante t .

¿Por qué NumPy y no OOP?

Por el Principio de Localidad de Caché (DOD). Un objeto por celda fragmenta la RAM. Con tensores `int8/int16` contiguos, la CPU evalúa 4800 celdas en registros SIMD simultáneamente.

¿Por qué la convección es estocástica?

Para evitar inyección masiva de momento no conservada. El 30 % de probabilidad redistribuye flujos existentes sin crear energía de la nada.

Conclusiones y Trabajo Futuro

Logros

- ▶ Validación del paradigma DOD para autómatas celulares masivos.
- ▶ Motor físico con $\mathcal{O}(N \cdot M)$ garantizado por frame.
- ▶ Termodinámica completa acoplada al flujo vectorial.

Trabajo Futuro

- ▶ **D2Q9**: diagonales para turbulencia y vorticidad.
- ▶ **CuPy / PyTorch**: migración a GPU para resoluciones superiores.
- ▶ **PyQt6**: GUI desacoplada con zoom, paneo y métricas en tiempo real.

¿Preguntas?

Grupo 1